

# KOPI-TWIN - Team15

---

## KOPI-TWIN - Team15 (*student proposal*)

---

**Assessment:** ✓ Fit · **Category:** Education + Training · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval · **Runner-up category:** Immersive Experience

**Proposal (as submitted):** *Kopi Twin* is a mobile **augmented-reality training app** that turns an ordinary table into a simulated kopitiam drink workstation for **new food-service employees** — trainees practise preparing and serving orders without occupying a stall or using real ingredients. Unlike video tutorials or fixed step-by-step guides, it uses a **real-time simulation** to generate customer orders, manage virtual ingredients, track recipe states and introduce changing workplace conditions. Trainees do guided lessons, then timed **peak-hour scenarios** balancing speed, accuracy and waste; performance is scored on procedural accuracy, completion time, incorrect actions and simulated ingredient usage. The POC focuses on traditional kopi, but the data-driven training system can extend to other workstations. Named technical features: a **native C++ digital-twin core on ARCore's NDK (C API)** converting a live camera scan into a persistent, bounded spatial-anchor map that survives across sessions; a **data-driven object system** (real-world anchors carry swappable behaviour definitions in data files, so the same engine runs different scenarios without code changes — swap JSON/txt instead of hard-coding); **user interaction with the object system** (handle objects via a button press); and a **bounded-space digital-twin** (a defined playable area with persistent anchor placement). Loop: a local native C++ update loop drives the AR session every frame — pull ARCore pose/plane data, update anchored virtual-object states, run the interaction/grading logic — performant enough for a live camera feed on-device. Tool: a **content/scenario editor** letting a non-programmer (a recipe designer or safety trainer) define anchor points, object behaviours and pass/fail interaction rules as data. Design features: guided AR training (instructions beside the relevant virtual equipment); peak-hour simulation (simultaneous orders, limited ingredients, time pressure); immediate error feedback (wrong ingredients/quantities/sequences/skipped steps); and a

performance & competency dashboard (accuracy, completion time, compliance, simulated waste).

*Note on category:* filed under **Education + Training** — the product is a **training simulator with learning objectives and measurable competency** (guided lessons → assessed peak-hour scenarios → a competency dashboard); the pedagogy is the product. Runner-up **Immersive Experience** (the AR medium) — but the goal is measurable skill acquisition, not atmosphere, so Education + Training is the better home. (Not Simulator: there is no system-modelling/physics simulation as the product — the simulation exists to train a procedure.)

**Scope & risk note (advisory, not a blocking condition):** persistent cross-session AR anchors + a bounded digital twin + a data-driven scenario system + peak-hour simulation + a scenario editor + a competency dashboard is a **large surface**, and this is a genuine **3D/AR** project (the "strong students only" row); the persistent-anchor AR core on the ARCore NDK is the key risk. Scope the floor to **the kopi POC on one bounded table** with the C++ AR loop, the data-driven object system, one guided lesson and the grading, and treat cross-session anchor persistence, peak-hour and the full dashboard as the stretch. (ARCore's NDK C API is a low-level platform API — permitted, like SDL/OpenGL ES — not a disallowed engine.)

*References:*

- [Bartender VR Simulator on Steam](#) (a drink-preparation **training simulator**: learn recipes in guided modes, then timed orders scored on accuracy and speed, with a completion certificate and a "Pro Mode" — the food-service drink-prep training precedent)
- [Diageo Bar Academy](#) (an industry bartending **training platform** with accredited skills courses and a virtual-bar experience — the real-world food-service training-with-competency precedent)
- [ARCore SDK for Android NDK \(C\)](#) — Google (the official native C API for camera pose/plane tracking and persistent anchors that the digital-twin core is built on — the AR-anchor tech precedent)

## Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

| Core Tech Requirement                                       | KOPI-TWIN |
|-------------------------------------------------------------|-----------|
| External Database                                           | ✓         |
| Authentication »                                            | ✓         |
| Analytics »                                                 | ●         |
| Real-time C++ Simulation Loop and/or C++ Performant Service | ✓         |
| Application Communication                                   | ✓         |
| Continuous Integration                                      | ✓         |
| Data-Driven Architecture                                    | ✓         |
| Front-end + Back-end                                        | ✓         |
| Custom Tools Development                                    | ✓         |
| 2D Graphical Output                                         | ✓         |
| 3D (strong students only)                                   | ✓         |
| Networking System                                           | ●         |

### How each requirement applies:

*Platform fit:* **Main:** Native mobile (Native Android — ARCore NDK on-device; device: Pixel 9a) — a camera-driven AR app; the Native PC build is the development/tools vehicle (the scenario editor). **Extended:** Firebase + a Linux server the team sets up by M2 (full use T2) for accounts, progress sync and the trainer/competency dashboard data.

**Front end:** The AR camera view with virtual kopitiam objects rendered over the live feed (anchored cups, ingredients, equipment), the guided-instruction

overlays, the order/timer/error-feedback HUD and the competency dashboard.

**Languages:** Native — Kotlin Native Android UI + the C++ engine (and the ARCore NDK C API) via JNI (Android Studio); OpenGL ES for the AR object rendering in a GLSurfaceView; Web/desktop — SDL2/3 + OpenGL ES for the PC dev/scenario-editor build.

**Back end:** The native C++ per-frame loop (pull ARCore pose/plane → update anchored virtual-object states → run the order/recipe simulation + interaction/grading), the data-driven object/behaviour system, and the bounded persistent-anchor map; plus accounts/progress/dashboard data. **Languages:** C++ (loop + simulation + grading); Firebase-backed services on the Extended tier.

**Backend scope:** Local now, server-ready (T2) — the accounts / progress-sync / trainer-dashboard seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the C++ loop is load-bearing — every frame it pulls ARCore pose/plane data, updates the anchored virtual-object states (order queue, virtual ingredients, recipe states, changing conditions), and runs the interaction + grading logic, sustaining a live camera feed on-device.
- *Data-driven:* customer orders, recipes/recipe-states, the per-anchor object behaviours, scenario definitions and the pass/fail grading rules are all data (JSON/txt), so a new drink, scenario or even a different workstation is a data change — not engine code.
- *2D:* the guided-instruction overlays, order/timer/error HUD and the competency dashboard are custom 2D rendering — satisfying the 2D gate.  
*3D:* ✓ **central** — the anchored virtual objects rendered in AR world space + the persistent bounded anchor map (strong-students; the key risk).  
*Networking:* progress/dashboard sync is async client↔cloud (Application Communication ✓), so a bespoke Networking System is optional (●).
- *Custom tool:* the **content/scenario editor** — a non-programmer defines anchor points, object behaviours and pass/fail interaction rules as data, without touching the C++ engine.
- *External DB:* accounts, progress history and the competency-dashboard data. *Auth* »: trainee/trainer accounts. *Analytics* »: optional — cross-trainee competency trends (the per-trainee dashboard is the team's own in-app analytics via G10).

## Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Education + Training pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

*M1:*

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the loop drives the **AR session** (ARCore pose/plane) + anchored object states + grading.

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a per-frame C++ loop that (on desktop) simulates the workstation without a camera for tooling/tests; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine and the ARCore NDK C API via JNI); OpenGL ES drawn in a GLSurfaceView over the live camera feed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the anchored object system, interaction (M1G5) and simulation (M1G11) run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System** — the **swappable per-anchor behaviour definitions**.

Tech & dependencies

- **Tech:** An entity/component model where real-world anchors carry swappable behaviour definitions (JSON/txt): drinks, recipes/recipe-states, ingredient objects, scenario config and grading rules, hot-reloadable — a new scenario is a data change, not engine code.
- **Prerequisites:** G1 (loop & anchored-object hooks).
- **Feeds into:** M2G2 (the scenario editor), M1G11 (simulation consumes it), M1G13/M2 grading.

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui debug overlays (anchor map, object/recipe state), Tracy profiling of the pull-pose→update→grade loop to sustain a live camera feed on-device, Catch2 tests for the grading rules and data loading, on a GitHub Actions Windows+Android matrix.

- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening); the on-device budget M2G1 validates.
- **G4 2D Graphics & Interactive Content Rendering** (extended to **AR object rendering**) — the virtual kopitiam objects rendered over the camera + the instruction/HUD overlays.

Tech & dependencies

- **Tech: Web build:** A renderer for the anchored 3D virtual objects composited over the live camera (via the GLSurfaceView), plus the 2D guided-instruction overlays and order/timer/error HUD; OpenGL ES, GLSL, FreeType. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime AR is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G1 (loop & AR session).
- **Feeds into:** M2G4 (advanced feedback visuals + dashboard charts); every visible frame.
- **G5 Real-time Interaction Engine** — button interaction with the anchored objects, step timing and scoring (the coffee-order steps + immediate error feedback).

Tech & dependencies

- **Tech:** Touch/button capture mapped to object interactions (grab/pour/serve), step timing, and per-step scoring with immediate error detection (wrong ingredient/quantity/sequence/skipped step).
- **Prerequisites:** G1 (tick), G2 (object behaviours).
- **Feeds into:** The grading + the competency dashboard; M2G13-class assessment.
- **G11 Domain Simulation Core** — the kopi-prep simulation: order generation, virtual ingredients, recipe states and changing workplace conditions.

Tech & dependencies

- **Tech:** A C++ simulation generating customer orders, managing virtual ingredient inventory, tracking recipe states through the prep steps, and introducing changing conditions — all tunable from data.
- **Prerequisites:** G1 (tick), G2 (recipe/order data).
- **Feeds into:** M2G11 (peak-hour: simultaneous orders + limited ingredients + time pressure); the training scenarios.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation** — on-device AR performance + persistent anchors across sessions.

Tech & dependencies

- **Tech: Web build:** Tracy profiling of the AR loop under a live feed, ASan on MSVC, memory pools, and on-device validation (Pixel 9a) that the bounded anchor map persists across sessions; the Firebase/Linux-server seam (accounts + progress + dashboard) set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine + ARCore NDK via JNI); OpenGL ES drawn in a GLSurfaceView where realtime AR is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The Native-Android deliverable + a stable live-camera budget with persistent anchors.

- **G2 ★ Content Editor — Core (Scenario Editor)** — a non-programmer defines anchor points, object behaviours and pass/fail interaction rules as data.

Tech & dependencies

- **Tech:** Dear ImGui + the shared C++ renderer: place/define anchor points, author per-object behaviour definitions and the pass/fail grading rules, with round-trip JSON/txt serialization and a preview, usable by a recipe designer / safety trainer.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's OpenGL ES renderer (preview).
- **Prerequisites:** G2 (behaviour/scenario data model), M1G4 (rendering for preview).
- **Feeds into:** All scenario content; the future non-kopi workstations; the custom-tools showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android Debug builds, tests (grading rules, simulation, data loading) and packaging via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime AR is needed — no SDL on native, some screens are pure Native UI.

- **Prerequisites:** G3 (CI & test foundation).
- **Feeds into:** Release/showcase builds of every goal.
- **G7 Firebase — Accounts, Progress Sync & Teacher Dashboards** — trainee accounts, progress history and the trainer-facing view.  
Tech & dependencies
  - **Tech:** Firebase Auth accounts, Firestore for progress/attempt history synced across devices, and a trainer dashboard reading aggregate trainee results, via the REST API from C++.
  - **Prerequisites:** G2 (result/progress data model), M1G5 (scores to store).
  - **Feeds into:** The competency dashboard; trainer oversight.
- **G10 Analytics & Learning Insights** — the **performance & competency dashboard** (accuracy, completion time, compliance, simulated waste).  
Tech & dependencies
  - **Tech:** Per-attempt metrics captured from the grading (procedural accuracy, completion time, incorrect actions, ingredient usage/waste), aggregated into a competency dashboard with custom-rendered charts.
  - **Prerequisites:** M1G5 (the scored metrics), M2G7 (stored history).
  - **Feeds into:** The competency-dashboard design feature; the trainee/trainer feedback loop.
- **G11 Domain Simulation II** — the **peak-hour** scenario: simultaneous orders, limited ingredients and time pressure.  
Tech & dependencies
  - **Tech:** The simulation matured for concurrent orders, constrained ingredient inventory and a time-pressure model, with difficulty scaling from data (guided → peak-hour).
  - **Prerequisites:** M1G11 (simulation core).
  - **Feeds into:** The peak-hour design feature; the speed/accuracy/waste balance at showcase.

## Track Goals & Team Composition

**Team composition (set by instructor):** 6 CS + 1 Design + 0 Art (7) — the **instructional design surface is large** (the guided-lesson curriculum, the grading/competency rubric, the peak-hour scenario design, the error-feedback design and the wayfinding-free AR UX), and with **no artist** the single designer owns all of that Design + UI/UX *and* takes the **D11 conditional** as a real graded Design goal each milestone (M1 Art Proof of Concept / M2 Art Implementation Quality — the last Design goal each milestone), which consumes one Design+UI slot per milestone (rules: min 1 Design mandated, max 9 total,  $D+A \leq 3$ ; see [team-formation.pdf](#)). **Load + art-risk flag:** this is a heavy squeeze — the instructional/UX design load is genuinely large for one designer, and the AR kopitiam objects + instruction/HUD/dashboard are a real art surface, so 0 artists is a serious risk. Strongly consider a **2nd designer** (to carry the instructional/UX load) and/or an artist within the  $D+A \leq 3$  cap; if the team stays 6 CS + 1 Design + 0 Art, scope the lessons/AR-object set hard and lean on a reusable style under D11 (a solo designer + 0 artists is the case the design pool flags). Note: for a training utility the audio floor is lighter (feedback cues + ambient kopitiam) — a lecturer waiver on heavy BGM may apply.

**Track goals** — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (learning objectives + target learners (new food-service staff), the skill gap the app closes, the guided → peak-hour progression, feasibility against the AR/engine plan)
- M1D2 Prototype — one guided kopi lesson working end to end (place workstation → follow steps → get graded)
- M1D11 Art Proof of Concept \*(conditional — no artist; Design-owned)\* — the learning-product visual direction: the virtual kopitiam AR objects (cups/ingredients/equipment) style plus the instruction/HUD and competency-dashboard visuals; 2 mockups.  
\*(The artist's absence + the drop to one designer squeeze out the M1D3/D6/D9 instructional-design goals.)\*

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness

- M2D11 Art Implementation Quality \*(conditional — no artist; Design-owned)\* — the kopitiam AR objects, the instruction/HUD and the competency dashboard implemented in-engine, no placeholders.
- **UI/UX M1** (graded by the UI/UX instructor):
  - M1U1 ★ UI Standards & Wireframe Baseline — the AR instruction-overlay, order/timer/error HUD and dashboard standards; readable over a live camera feed.
- **UI/UX M2:**
  - M2U1 ★ Functional UI & Usability — lesson, AR session, result and dashboard navigable unaided on the Native-Android target.
  - M2U2 ★ UX Validation & Final Polished UI — validated, polished instruction + feedback UX tested with trainees on the Pixel 9a.
- **Art:** 0 artists — the art surface (the virtual kopitiam AR objects — cups/ingredients/equipment — plus the instruction/HUD and competency-dashboard visuals) is covered by the D11 conditionals, taken as Design goals by the single designer (see Design M1/M2). The AR-object + dashboard art is a real surface, so 0 artists is a serious risk — scope the AR-object set hard and lean on a reusable style.
- **TEAM M1:**
  - M1TA1 ★ Overall Audio Experience — initial audio from the week-5 needs list present & working (kopitiam ambience + interaction, correct/incorrect and order cues), suitable and correctly triggered (a lecturer waiver on heavy BGM may apply for this training utility).
  - M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback for overlapping feedback cues.
- **TEAM M2:**
  - M2TA1 ★ Overall Audio Experience — the Education-floor set: ambient + interaction / correct / incorrect / order-in / peak-hour cues, correctly triggered and clear under time pressure.
  - M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the playback functionality the build needs (multi-channel; distinct correct-vs-error cues).

- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the scenario editor), prototype demo, team post-mortem (wk14)
- 

*Generated by the Project Proposal Assessor skill · grounding: live folder  
(Project 2 Technical adjustments/ ) · references verified 2026-09-21.*